

ALGORITHMS, FLOWCHARTS AND PSEUDO CODE



Surbhi Chhabra
Assistant Professor, JKLU Jaipur

OVERVIEW

- We will explore simple algorithms using flowcharts and pseudo code
- We will look at:
 - Sequencing – putting things in the right order
 - Conditions – doing different things depending on some rules
 - Iteration (Looping) – repeating something

IMPORTANT DEFINITIONS

- The **algorithm** and **flowchart** are two types of tools to explain the process of a program.
- Writing a logical step-by-step method (procedure) to solve the problem is called the **algorithm**. In order to solve a mathematical or computer problem, this is the first step in the process.
- A procedure is a finite sequence of well-defined instructions, each of which can be carried out in a finite amount of time. By finite, it means we can count the number of instructions in the Procedure. And by well-defined we mean to say that there is no ambiguity in the instruction.
- An algorithm includes calculations, reasoning, and data processing. Algorithms can be presented by natural languages, pseudocode, and flowcharts, etc.
- A **flowchart** is the graphical or pictorial representation of an algorithm with the help of different symbols, shapes, and arrows to demonstrate a process or a program. As the name indicates, is about the flow of execution of our algorithm. Graphically depict the logical steps to carry out a task and show how the steps relate to each other.
- A flowchart consists of special geometric symbols connected by arrows. Within each symbol is a phrase presenting the activity at that step. The shape of the symbol indicates the type of operation that is to occur. For instance, the parallelogram denotes input or output. The arrows connecting the symbols, called **flowlines**, show the progression in which the steps take place. Flowcharts should “flow” from the top of the page to the bottom.

EXAMPLES: ALGORITHM

For instance, the algorithm to find the sum of two numbers is

- Take two numbers
- Add the numbers
- Display the result

Or the algorithm to make a cup of Darjeeling tea is

- Take 1 tea spoon of Darjeeling tea leaves and place it inside a tea pot
- Pour some water
- Cover the pot with the lid and boil
- Let it brew for about 3 to 5 minutes
- Strain the tea in a cup and serve

Write the algorithm to log into your Facebook account in simple English as:

- Go to www.facebook.com
- Enter Email ID and Password
- Click on Log In button

ALGORITHM VS FLOWCHART

Algorithm	Flowchart
It is a procedure for solving problems.	It is a graphic representation of a process.
The process is shown in step-by-step instruction.	The process is shown in block-by-block information diagram.
It is complex and difficult to understand.	It is intuitive and easy to understand.
It is convenient to debug errors.	It is hard to debug errors.
The solution is showcased in natural language.	The solution is showcased in pictorial format.
It is somewhat easier to solve complex problem.	It is hard to solve complex problem.
It costs more time to create an algorithm.	It costs less time to create a flowchart.

WHAT IS PSEUDO CODE?

- Pseudo code is an informal, high-level, natural-language description of an algorithm. Pseudo code uses English-like phrases with some Python terms to outline the task.
- Pseudocode has several advantages. It is compact and probably will not extend for many pages as flowcharts commonly do. Also, the pseudocode looks like the code to be written and so is preferred by many programmers.
- Pseudocode allows the programmer to focus on the steps required to solve a problem rather than on how to use the computer language. The programmer can describe the algorithm in Python-like form without being restricted by the rules of Python. When the pseudocode is completed, it can be easily translated into the Python language. E.g.

Start

get value

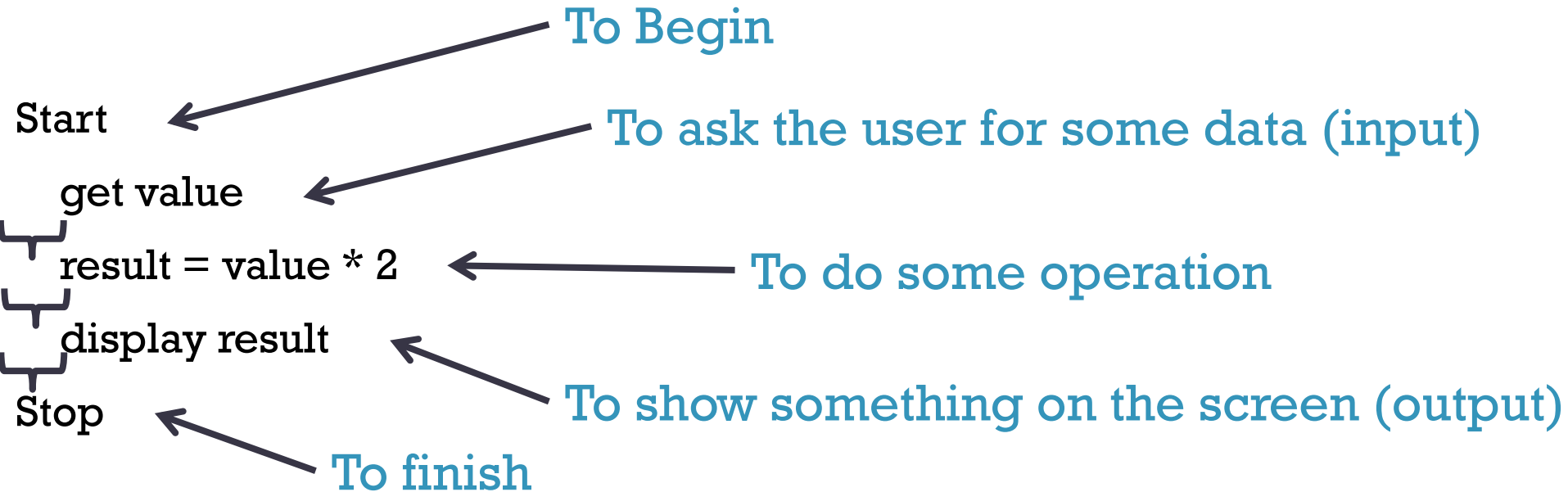
result = value * 100

display result

Stop

PSEUDO CODE

- Pseudo code is informal but we will use this basic structure:



The indentation is important...

WHAT IS A FLOWCHART?

- A flowchart is a graphical representation of an algorithm
- It uses standard shapes including:



Terminator



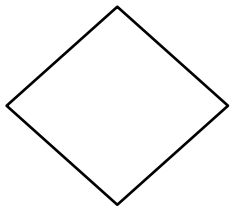
Input or output



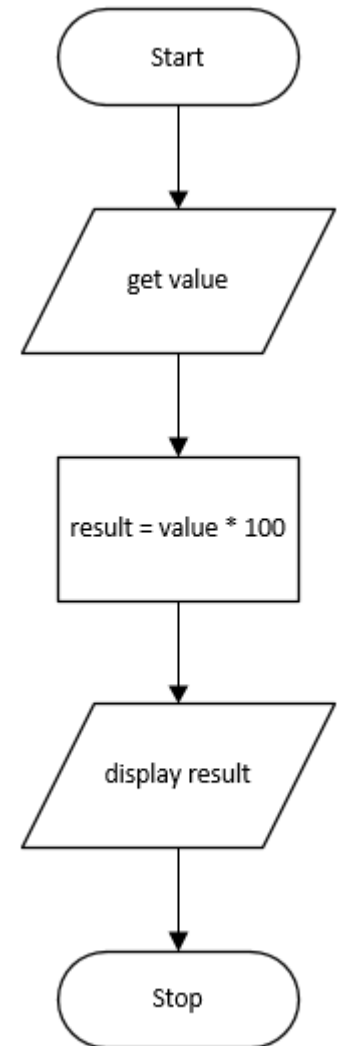
Flow line



Process

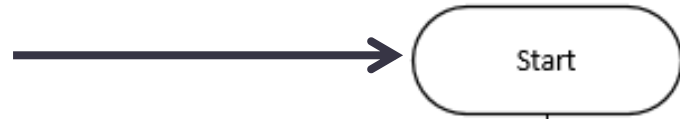


Decision

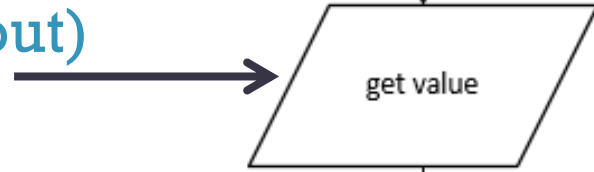


FLOWCHART

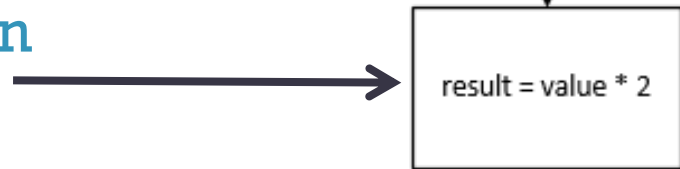
To Begin



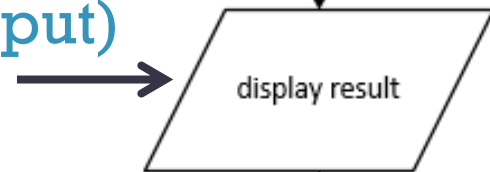
To ask the user for some data (input)



To do some operation



To show something on the screen (output)



To Finish



WHY USE PSEUDO CODE OR FLOWCHARTS?

- They are more **concise** and **clear** than a paragraph of writing
- They show the logical branching with **indentation** or **flow lines**
- They are **not specific** to a particular programming language
- They can be used to **plan** a solution before coding it
- They can be **checked** by or discussed with someone who cannot program

SIMPLE SEQUENCING

- Problem: calculate the average of three input numbers.
- How do we do this?

Start

get n1

get n2

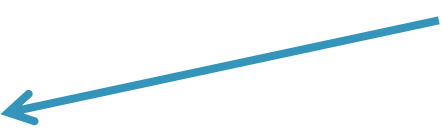
get n3

average = (n1 + n2 + n3)/3

display average

Stop

n1 represents the first number entered

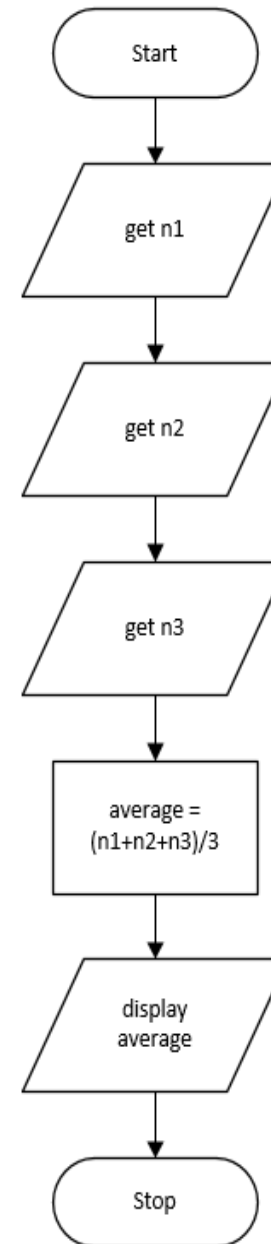


average will contain the result
of the sum: (n1 + n2 + n3) / 3



SIMPLE SEQUENCING

- Here is the flowchart for the same problem
- Remember to use the right shapes
- for input or output 
- for a process or calculation 



CONDITIONS

- Conditions are the way we make a choice between two or more alternatives.
- Doing something different depending on a condition is called **altering the flow of control**.

Start

get mark

if mark $\geq$ 50  1. Evaluate the condition getting True or False

 result = "Pass"  2. If the result is True then perform these operation(s)

else

 result = "Fail"  3. If it is False then perform these operation(s)

end-if

display result

Stop

A NOTE ON META-SYMBOLS

- Note that the `<` and `>` characters are called meta-symbols
- What lies between then indicates the **kind of thing** which is allowed in its place.

if `<condition>` then
 `<operations for then-part>`
else
 `<operations for else-part>`
end-if



`<variable> = <expression>`



- basic + bonus
- 120 + 10

CONDITIONS

- Problem: Work out whether the student has passed or failed

Start

get mark

if mark $\geq$ 50

 result = "Pass"

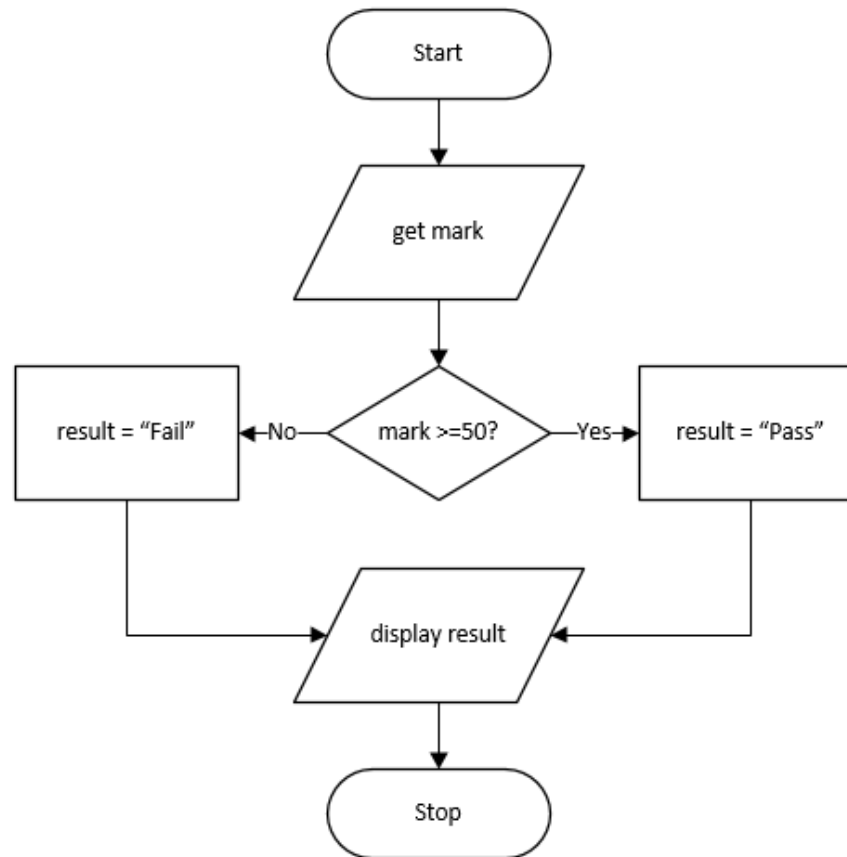
else

 result = "Fail"

end-if

display result

Stop



CONDITIONS

- There can be more than one expression in the <then> or <else> parts. E.g.

Start

Get mark

if mark $\geq$ 50 then

 result = "Pass"

 message = "Well done!"

else

 result = "Fail"

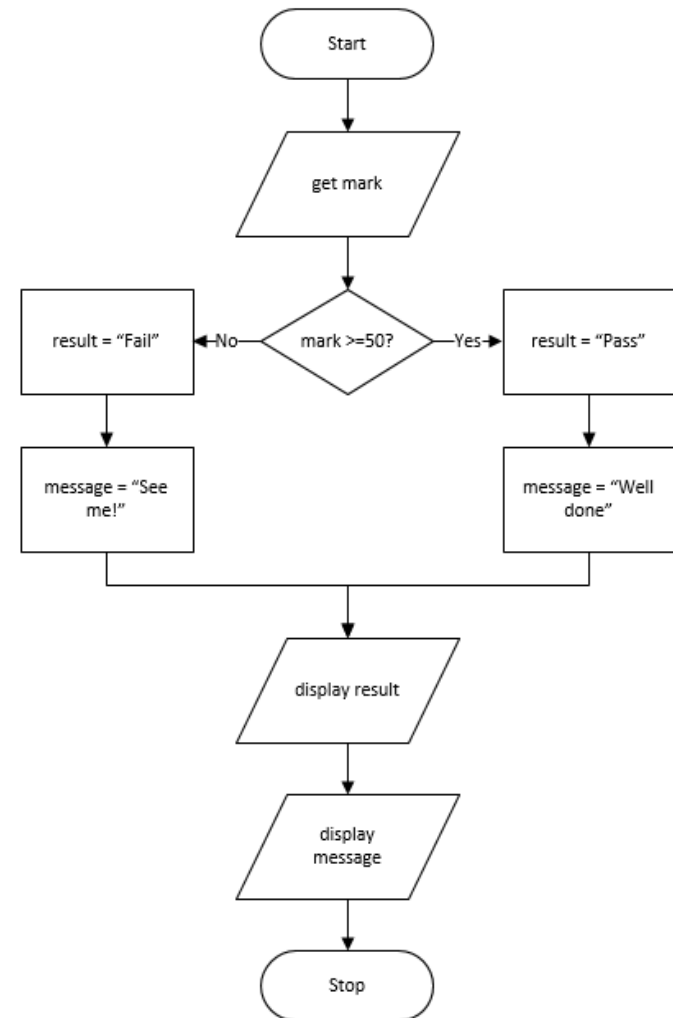
 message = "See me!"

end-if

display result

display message

Stop



CONDITIONS

- It is also possible to have multiple conditions.

Start

get mark

if mark < 50 then

result = "Fail"

else if mark < 60 then

result = "Pass"

else if mark < 70 then

result = "Merit"

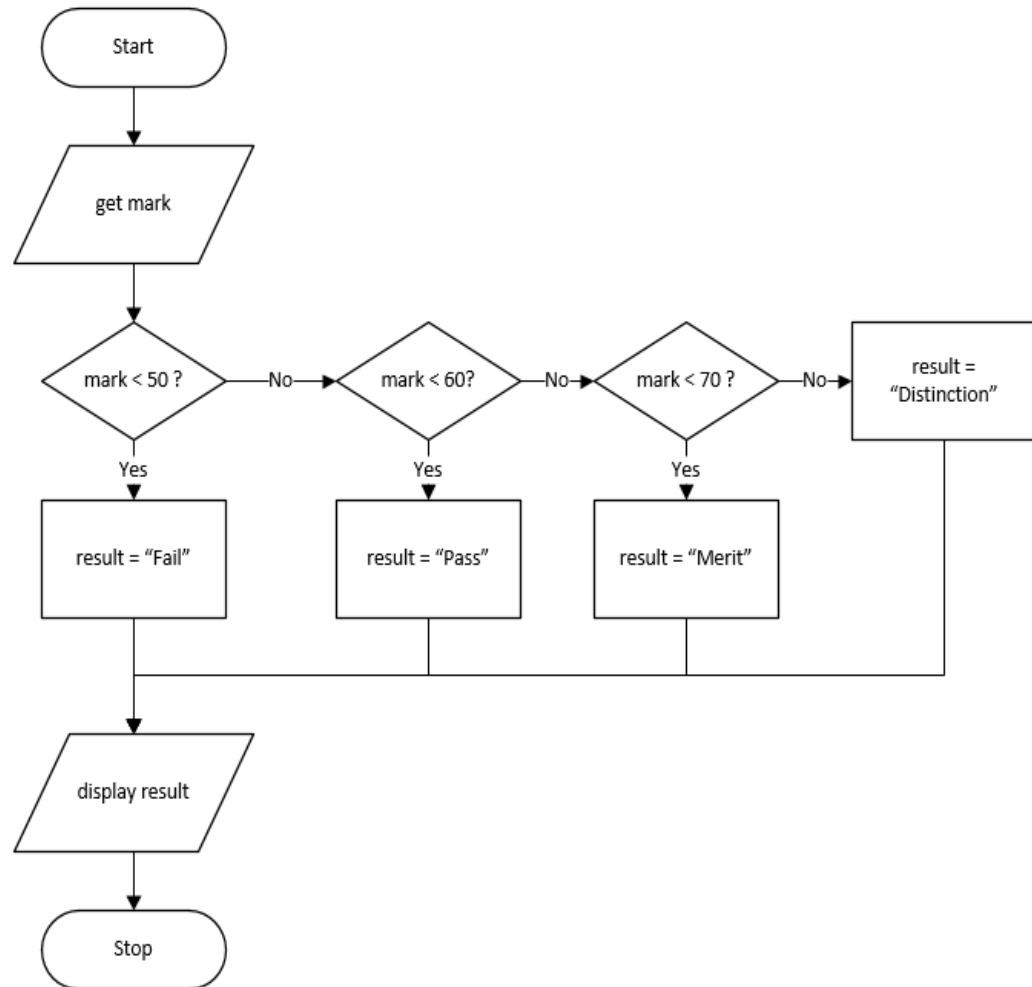
else

result = "Distinction"

end-if

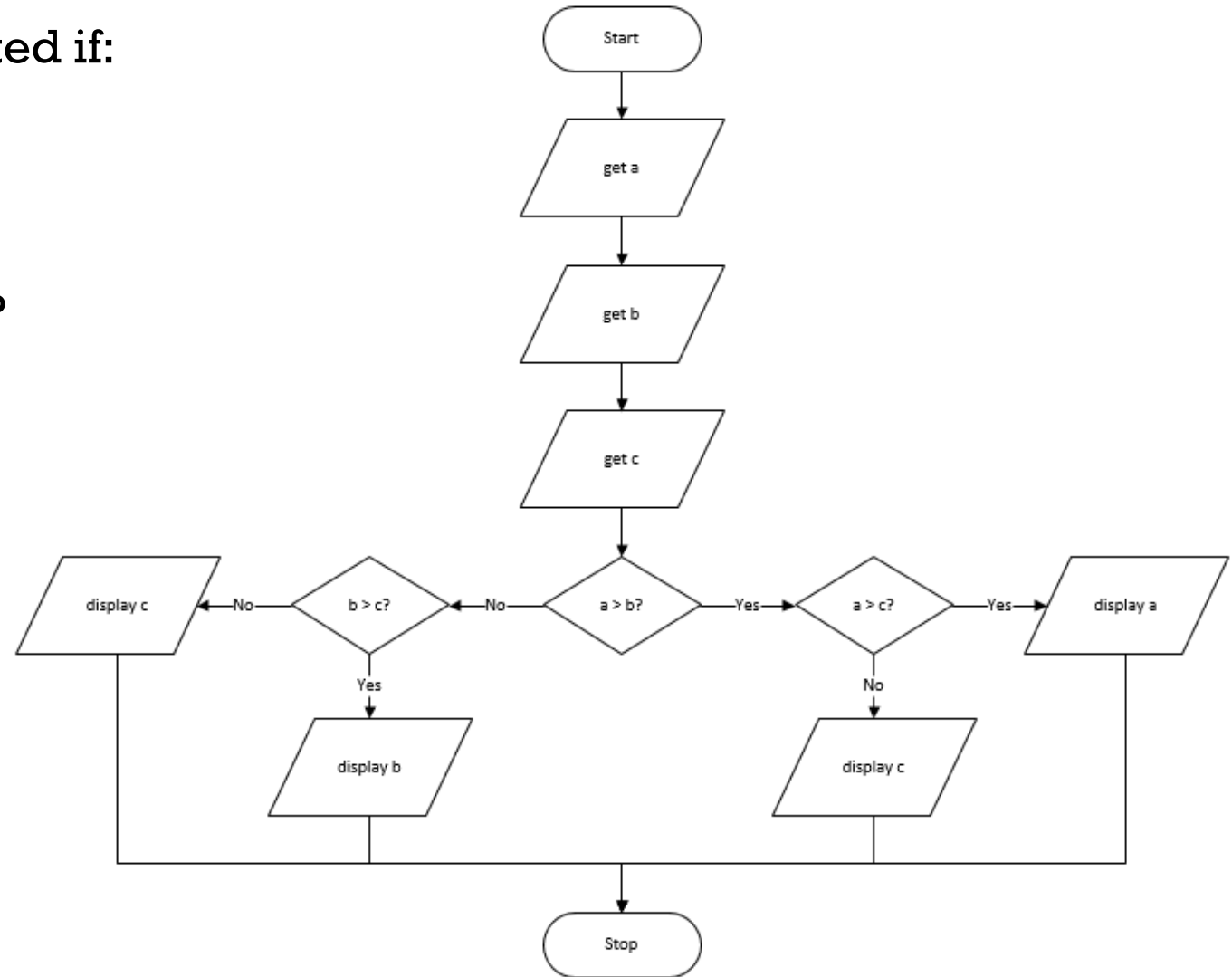
display result

Stop



WHAT DOES THIS DO?

- What value is printed if:
 - a=5, b=7, c=2 **b**
 - a=8, b=2, c=4 **a**
 - a=3, b=5, c=8 **c**
- What is this doing?



YOUR TURN

Write some pseudo code and sketch a flow chart for the following cinema ticketing system:

- If customer is aged under 12 then the ticket £5
- Otherwise, if they are over 65 the ticket is £7
- Otherwise the ticket is £10

PSEUDO CODE

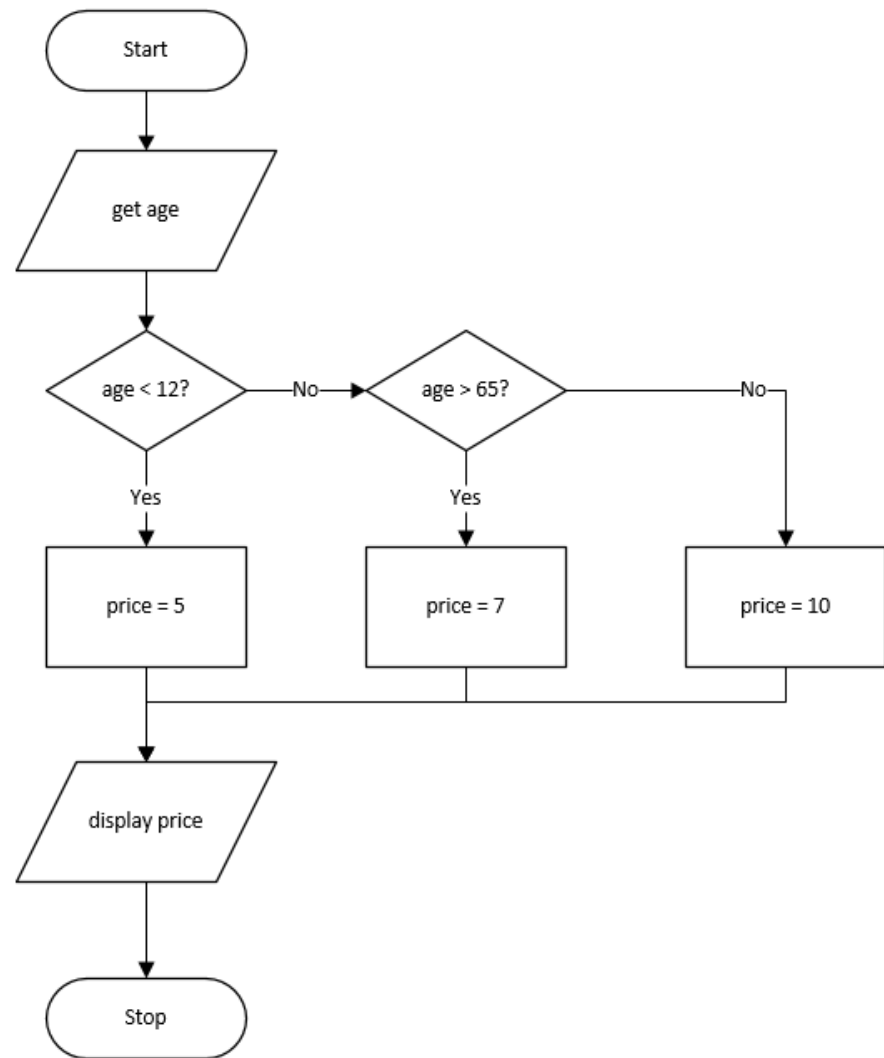
FLOWCHART

```
Start
  get age

  if age < 12:
    price = 5
  else if age > 65:
    price = 7
  else
    price = 10
  end-if

  display price

Stop
```



ITERATION

- If you wanted to display all the numbers from 1 to 1000, you wouldn't want to do this, would you?

Start

display 1

display 2

etc.

Stop

- Wouldn't it be better to say something like this?

number = 1

while the number <= 1000...

display the number

increase the number

ITERATION: LOOP

- A programming structure that executes instructions many times is called a **repetition structure** or a **loop structure**.
- Loop structures need a test (or condition) to tell when the loop should end. Without an exit condition, the loop would repeat endlessly (an infinite loop).
- One way to control the number of times a loop repeats (often referred to as the number of passes or iterations) is to check a condition before each pass through the loop and continue executing the loop as long as the condition is true.

ITERATION: DISPLAY NUMBERS UP TO 1000

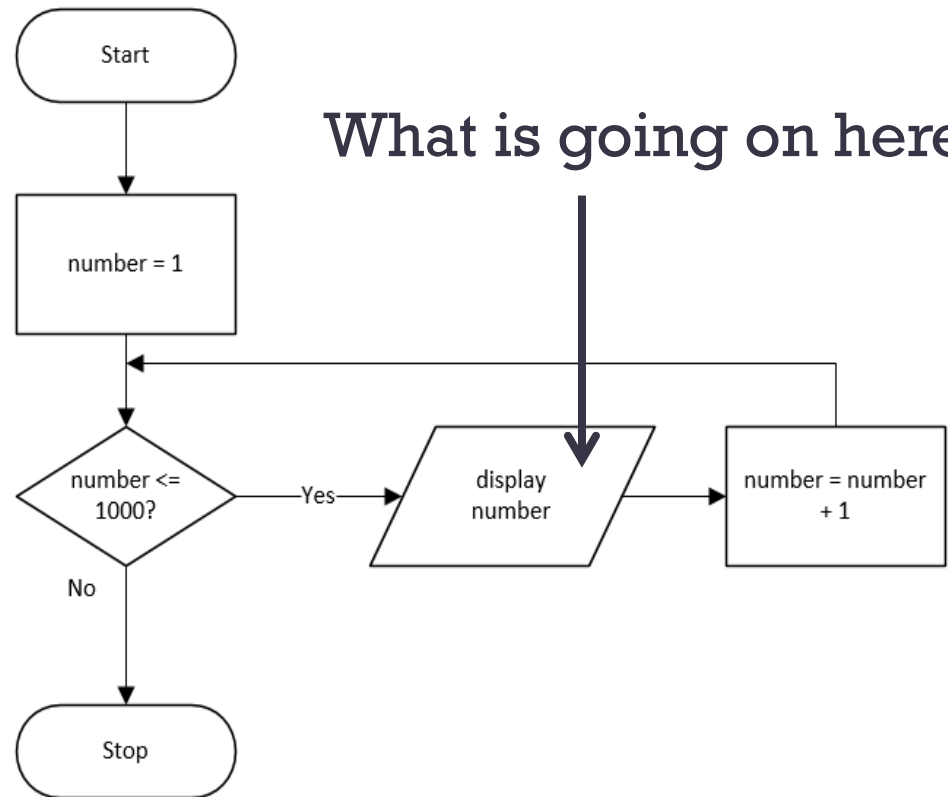
- We can write this in pseudo code:

Start

```
number = 1
while number <= 1000
    display number
    number = number + 1
end-while
```

Stop

Why do we
need to do this?



ITERATION: GET FIVE NUMBERS

- Problem: Allow the user to enter 5 numbers
- *How do we know when we have 5 numbers?*

Start

```
count = 0
```

```
while count < 5
```

← Why less than?

```
    get number
```

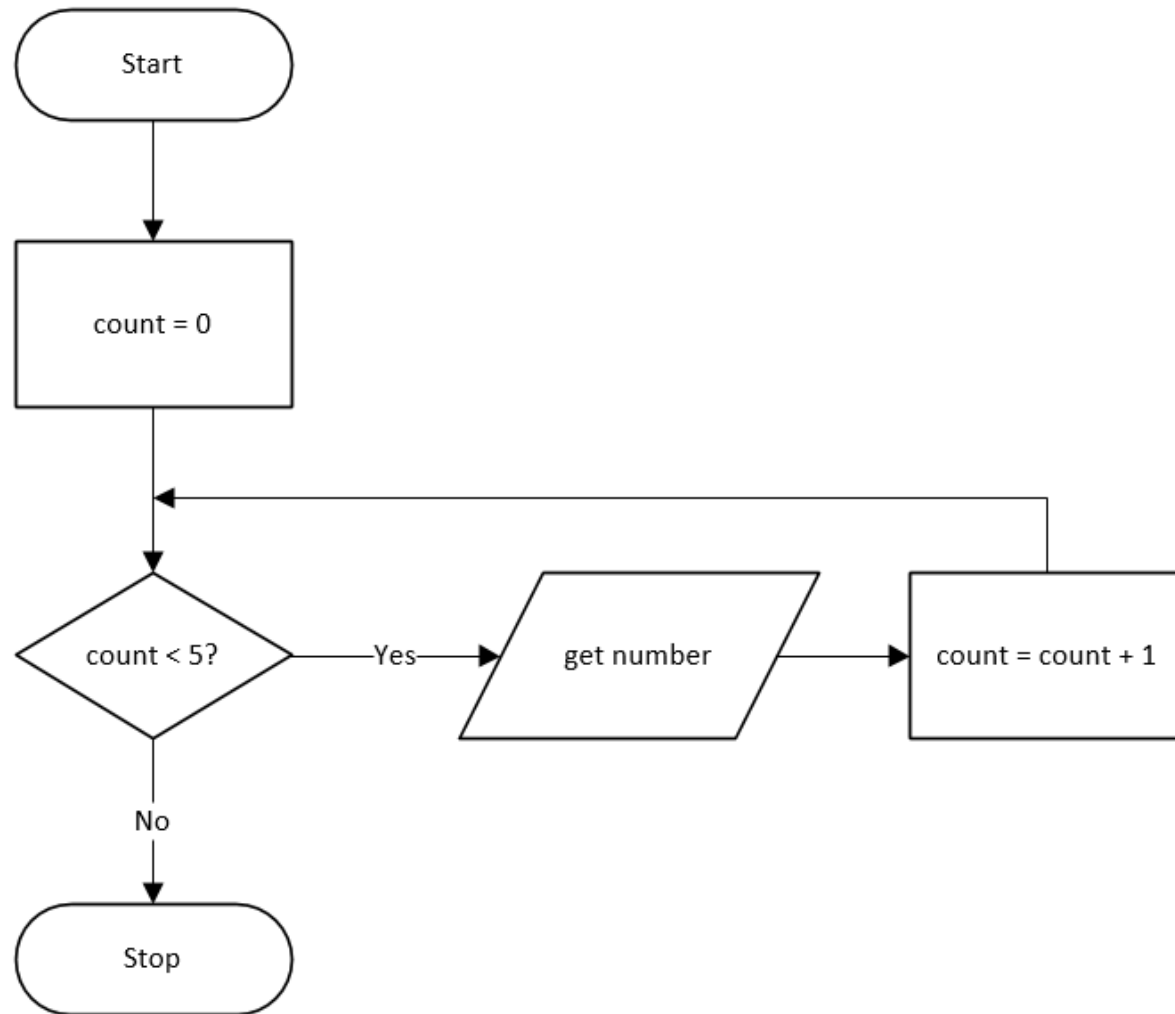
```
    count = count + 1
```

← Why do we need to do this?

```
end-while
```

Stop

ITERATION: GET FIVE NUMBERS



ITERATION: TOTAL FIVE NUMBERS

- Now let's also display the total of those numbers

Start

Here is the pseudo code to ask the user for five numbers

```
count = 0
```

```
total = 0 ← We need a total, initially set to zero
```

```
while count < 5
```

```
    get number
```

```
    count = count + 1
```

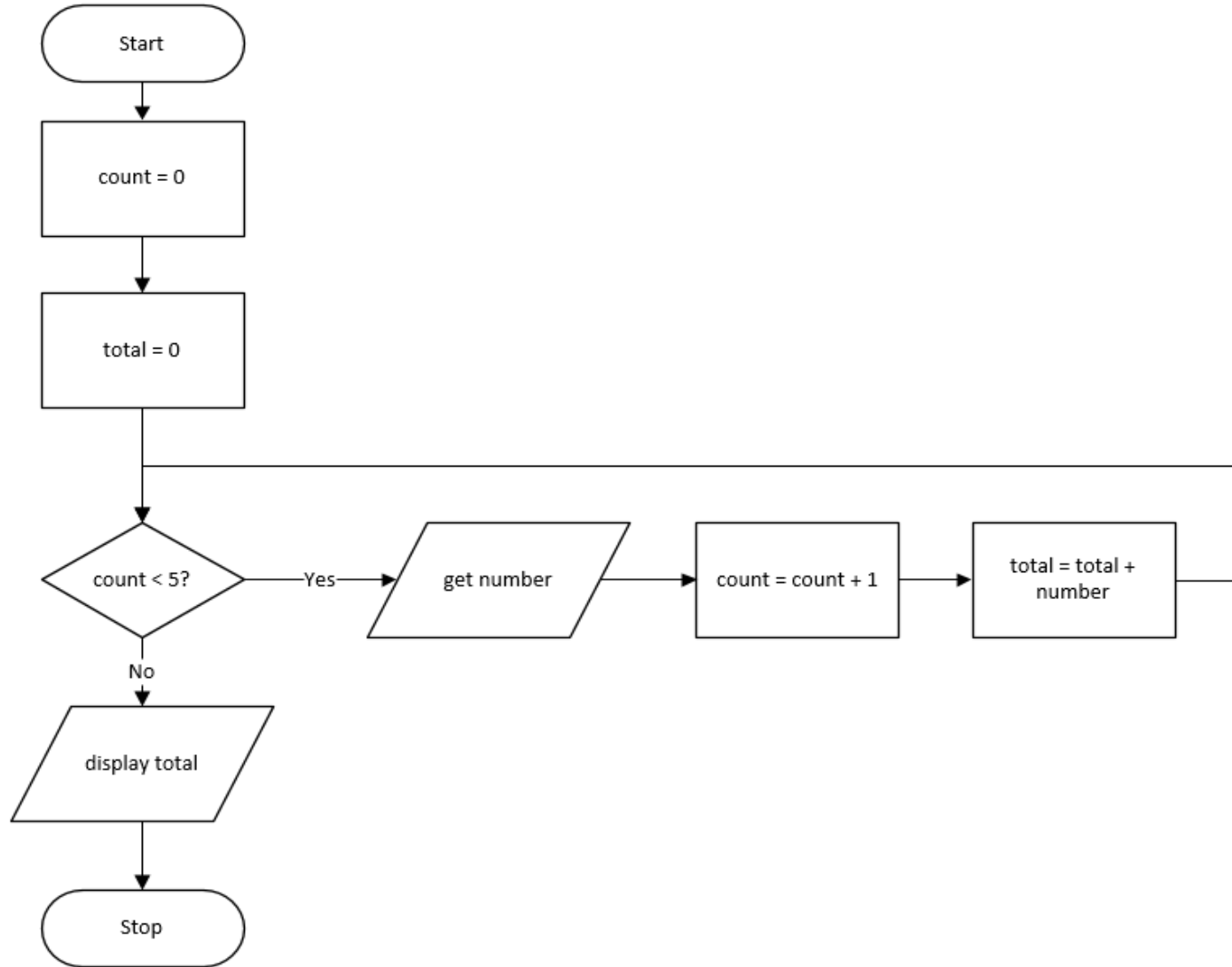
```
    total = total + number ← We need to add the number to that total
```

```
end-while
```

```
display total ← At the end, we need to display the final total
```

Stop

ITERATION: TOTAL FIVE NUMBERS

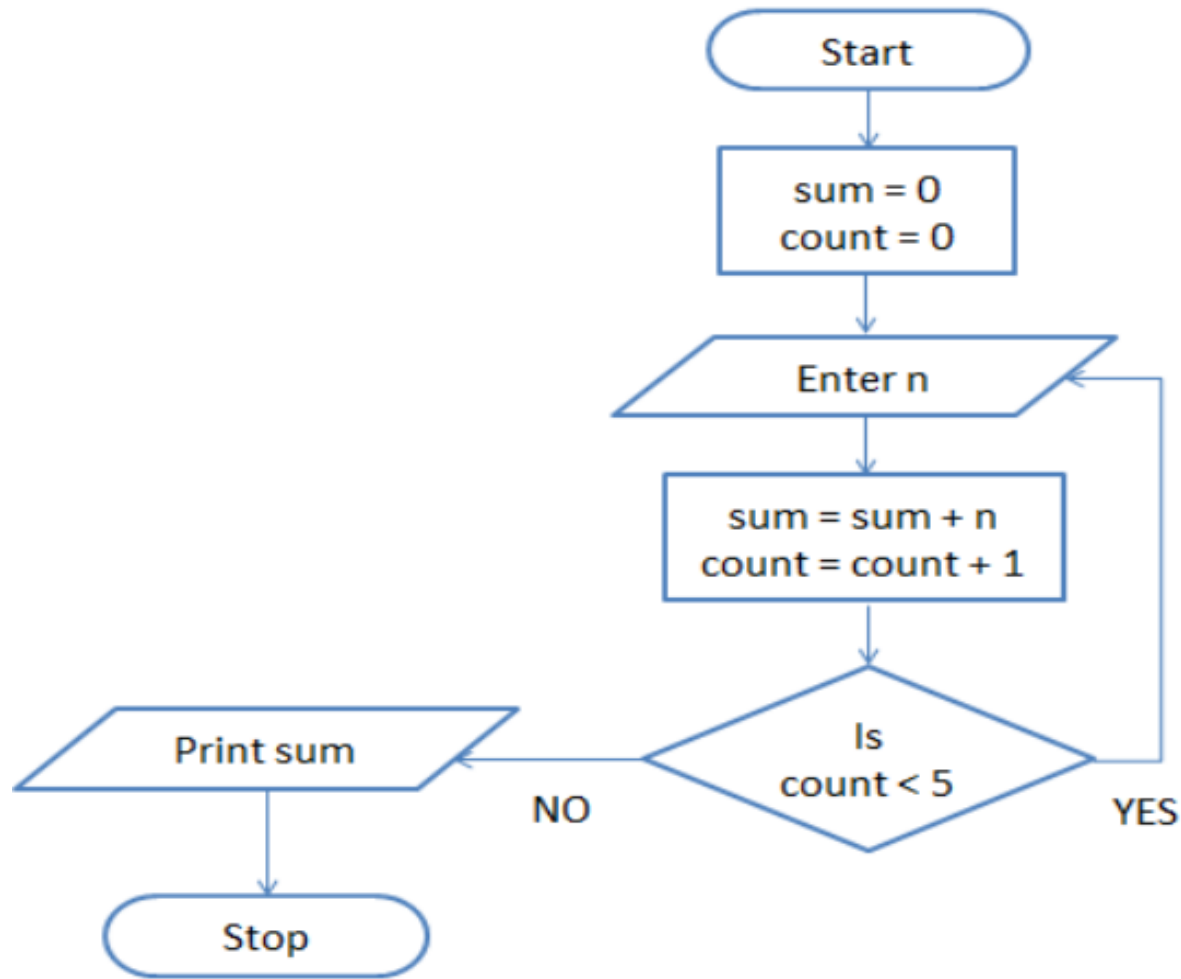


ITERATION: TOTAL FIVE NUMBERS: SECOND METHOD

Algorithm (in simple English)

- Initialize $\text{sum} = 0$ and $\text{count} = 0$ (PROCESS)
- Enter n (I/O)
- Find $\text{sum} + n$ and assign it to sum and then increment count by 1 (PROCESS)
- Is $\text{count} < 5$ (DECISION)
- if YES go to step 2
else
Print sum (I/O)

ITERATION: TOTAL FIVE NUMBERS: SECOND METHOD



YOUR TURN

- Write some pseudo code and sketch a flow chart to:
 - Ask the user for ten numbers
 - Display the total and average of these numbers

AVERAGE TEN NUMBERS PSEUDO CODE

Start

count = 0

total = 0

while count < 10

 get number

 count = count + 1

 total = total + number

end-while

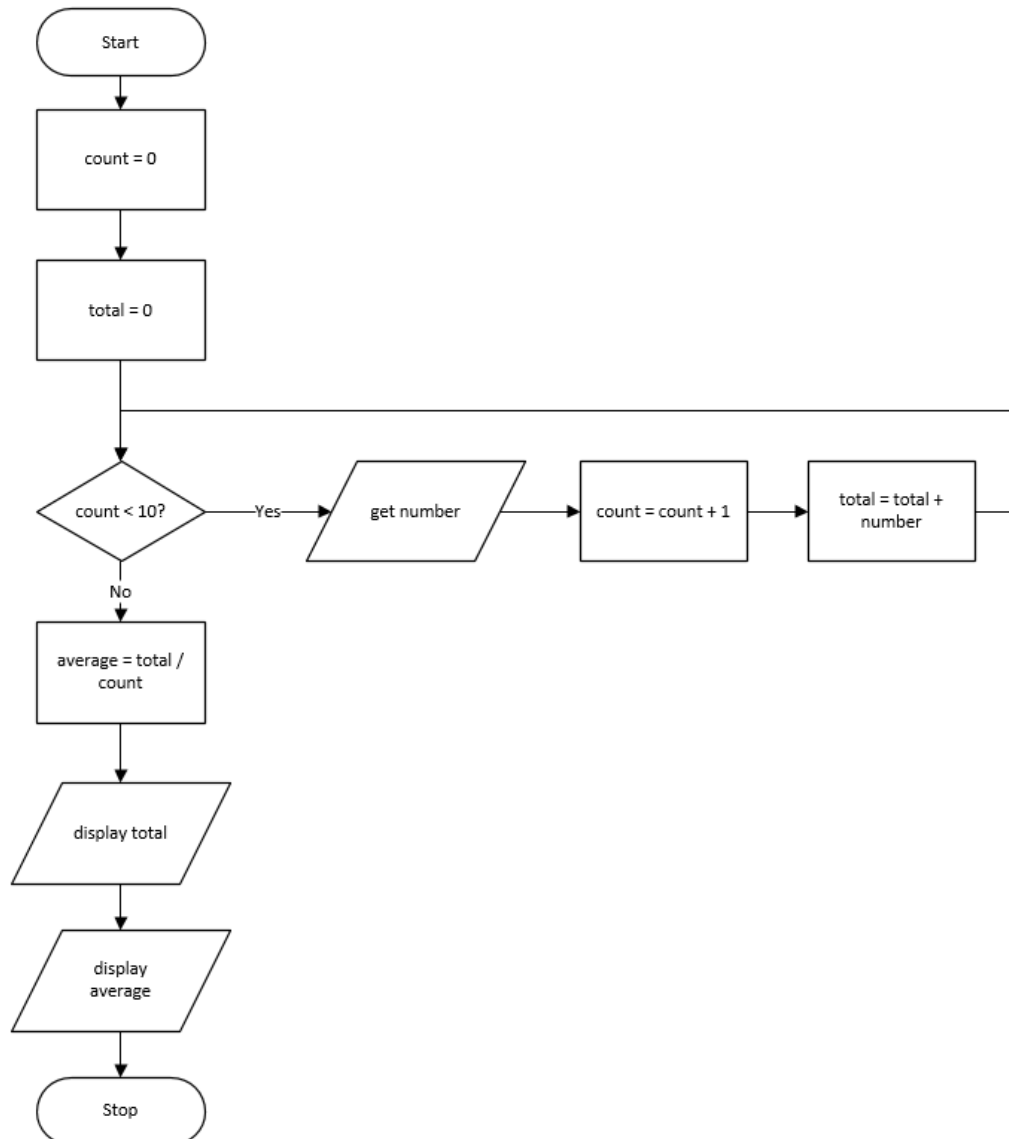
average = total / count

display total

display average

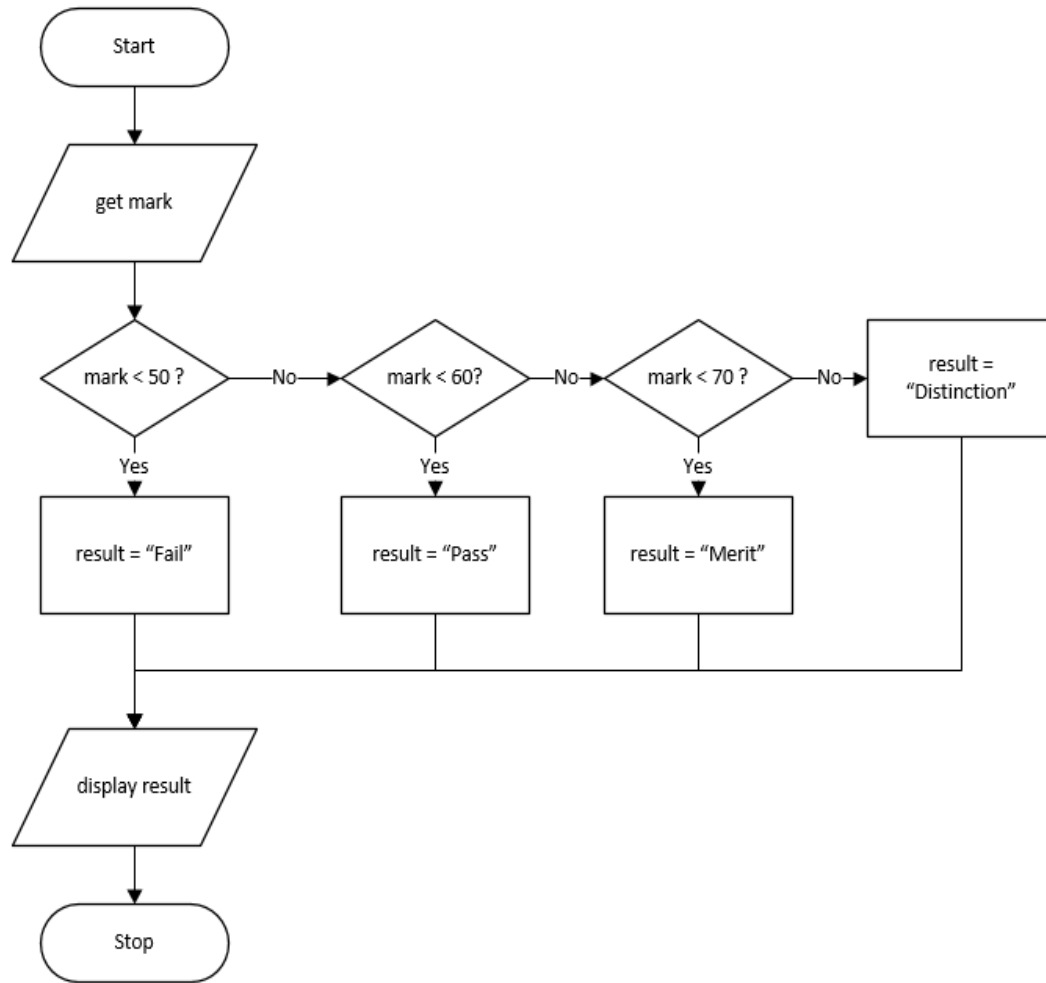
Stop

AVERAGE TEN NUMBERS PSEUDO CODE



TRACING A FLOWCHART

- Use a table of possible values and expected outcomes



Mark	Expected Result	Actual Result
49	Fail	Fail ✓
50	Pass	Pass ✓
59	Pass	Pass ✓
60	Merit	Merit ✓
69	Merit	Merit ✓
70	Distinction	Distinction ✓

You must carefully consider the test values because you need to validate your algorithm

WHILE LOOP WITH ELSE

- While statement in python also supports else statement in association with it.
- The else statement written after the while statement will be executed when the expression of while returns False.
- Syntax:

while expression:

Statement(s)

else:

Statement(s)

```
count = 0;
```

```
while count < 10:
```

```
    print("My name is Surbhi")
```

```
    count += 1
```

```
else:
```

```
    print("String is printed ten times!")
```

ITERATION TYPES

- There are different ways of expressing iteration using pseudo code
- Which you use depends on what you are trying to do

```
while <condition>  
    <operation(s)>  
end-while
```

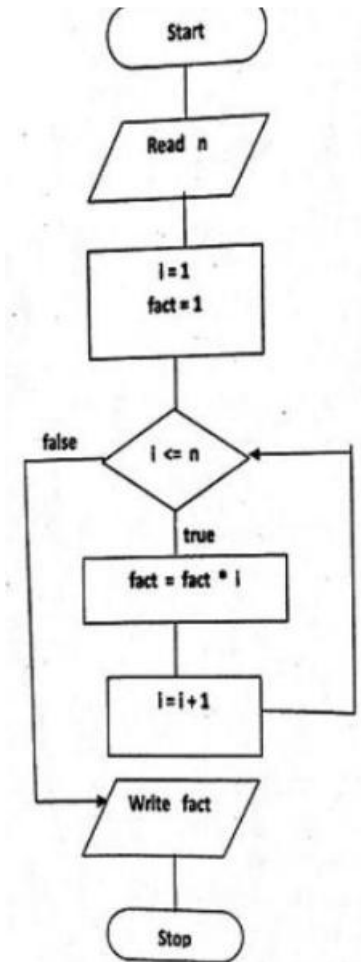
```
do  
    <operation(s)>  
while <condition>
```

```
repeat  
    <operation(s)>  
until <condition>
```

- The programming language you intend to use is also likely to affect your choice
- We will use the first one because it translates most directly to Python

EXAMPLE: FACTORIAL OF A NUMBER

- Step 1 : Start
- Step 2 : Read/Enter/Input n
- Step 3 : Initialize counter variable i to 1 and fact to 1
(i=1, fact=1)
- Step 4 : if $i \leq n$ go to step 5 otherwise goto step 7
- Step 5 : calculate $\text{fact} = \text{fact} * i$
- Step 6 : increment counter variable i and goto step 4
- Step 7 : Write/Display/Print fact.
- Step 8 : Stop



EXAMPLE: FACTORIAL OF A NUMBER : REPEAT UNTIL

- Step 1: Start
- Step 2: Declare variables n, factorial and i.
- Step 3: Initialize variables
 - $\text{factorial} \leftarrow 1$
 - $i \leftarrow 1$
- Step 4: Read value of n
- Step 5: Repeat the steps until $i = n$
 - 5.1: $\text{factorial} \leftarrow \text{factorial} * i$
 - 5.2: $i \leftarrow i + 1$
- Step 6: Display factorial
- Step 7: Stop

SUMMARY

- We have looked at what is algorithm, pseudo code and flowcharts
- We have looked at why pseudo code and flowcharts are useful
- We have looked at pseudo code and flowcharts for **sequencing**, **conditions** and **iteration**